

Trabalho Final (N3) — Portabilidade do App Aluno para React

Centro Universitário SATC — Engenharia de Software

Disciplina: Front-end

Professor: Lucas Beskow Vergara

Avaliação: N3 — Projeto Final em Dupla

Peso: 10,0

Objetivo

O objetivo deste trabalho é demonstrar a sua compreensão dos fundamentos da biblioteca **React** através da **portabilidade** (*porting*) da aplicação **App Aluno**, que construímos juntos ao longo das aulas de Fundamentos.

Durante as aulas de Fundamentos, desenvolvemos a aplicação em **HTML, CSS e JavaScript puro** (fluxo de autenticação + telas internas). Agora, na etapa de Frameworks e Bibliotecas, você deve **reescrever essa mesma aplicação em React**, aplicando todos os conceitos vistos nas Aulas 10 a 14: componentes, props, *state*, *hooks*, formulários controlados, roteamento, Context API e consumo de API.

A funcionalidade final deve ser equivalente à versão original. O que muda é a **stack**: de HTML/CSS/JS estático para uma **SPA em React**.

Ponto de partida

A versão original (referência da funcionalidade e do layout) está disponível em:

- **App Aluno (HTML/CSS — base das aulas):** <https://github.com/lucasbeskow/app-aluno>

O repositório contém:

- O fluxo de autenticação já pronto em HTML/CSS:
 - **Login** (`index.html`)
 - **Recuperar senha** (`recuperar-senha.html`)
 - **Nova senha** (`nova-senha.html`)
 - **Cadastro — passo 1** (`cadastro-usuario.html`)
 - **Cadastro — passo 2 / dados** (`cadastro-usuario-dados.html`)
 - **Dashboard** (`dashboard.html`)
- Os arquivos de estilo (`styles.css` , `mobile.css`).
- A pasta `telas/` com os *prints* de referência das telas internas:
 - `telas/Dashboard.png`
 - `telas/Disciplinas.png`
 - `telas/Perfil.png`
 - `telas/TutorIA.png`

Use o HTML/CSS existente e os *prints* como **referência visual**. Você não precisa reaproveitar o HTML diretamente — o esperado é **recriar a interface em JSX** com componentes.

Requisitos Funcionais

A aplicação portada para React deve ser capaz de:

1. **Fluxo de autenticação completo:** telas de Login, Recuperar senha, Nova senha e Cadastro (passo 1 e passo 2).
 2. **Navegação entre telas via rotas (SPA):** após o login, o usuário acessa as telas internas (Dashboard, Disciplinas, Perfil, TutorIA) sem recarregar a página inteira.
 3. **Dashboard** exibindo informações do usuário logado (ex.: nome, saudação, resumo).
 4. **Disciplinas:** listar as disciplinas a partir de uma coleção de dados (renderização de lista). A lista pode vir de um *array* local **ou** de uma API.
 5. **Perfil:** exibir os dados do usuário.
 6. **TutorIA (ou outra tela que consuma dados externos):** consumir dados de uma API e exibir os três estados de carregamento (carregando / erro / dados).
 7. **Formulários controlados** com validação simples (ex.: campo obrigatório, e-mail inválido) e exibição de mensagem de erro na tela.
 8. **Menu de navegação** com indicação visual da rota ativa.
-

Requisitos Técnicos Mínimos

A aplicação **deve obrigatoriamente** utilizar os conceitos abaixo. Cada um deles foi trabalhado em aula e está mapeado na tabela de avaliação.

1. **Projeto criado com Vite** (`npm create vite@latest`) — *Aula 10*.
2. **Componentização:** a UI deve ser dividida em **múltiplos componentes reutilizáveis e bem definidos** (ex.: `InputField`, `Card`, `Sidebar`, `Header`, `DisciplinaCard`) — *Aula 11/12*.
3. **Props e children:** uso de props para parametrizar componentes e da prop `children` para componentes "container" — *Aula 12*.
4. **Hooks — useState e useEffect:** gerenciamento de estado e efeitos colaterais — *Aulas 11 e 13*.
5. **Formulários controlados:** *inputs* vinculados ao estado (`value` + `onChange`) — *Aula 12*.
6. **Renderização condicional:** uso do operador `&&` (ou ternário) para exibir mensagens de erro / estados — *Aula 13*.
7. **Renderização de listas:** uso de `.map()` com a prop `key` obrigatória — *Aula 11*.
8. **Roteamento com React Router** (`react-router-dom`): `BrowserRouter`, `Routes`, `Route`, `NavLink` e `useNavigate` (redirecionamento após o login) — *Aula 14*.
9. **Context API:** compartilhar os dados do usuário entre telas (ex.: nome no Dashboard, no Perfil e na Sidebar) sem *prop drilling* — *Aula 14*.
10. **Consumo de API com fetch:** ao menos uma tela deve buscar dados em uma API externa, tratando os três estados (`loading` / `erro` / `data`) com o padrão `try / catch / finally`. Lembre-se de checar `res.ok` antes de parsear — *Aula 13*.
11. **Estilização com className** e arquivos `.css`, com layout **responsivo** (desktop e mobile) — *Aula 11*.

Sugestões para o consumo de API

A escolha da API é livre, desde que o padrão `loading/erro/data` seja respeitado. Algumas opções:

- A **GitHub API** (`GET https://api.github.com/users/{username}`), como vista em aula.
- Uma API de dados *fake* como **JSONPlaceholder** (`https://jsonplaceholder.typicode.com`) para simular disciplinas, tarefas ou avisos.
- Simular a resposta da **TutorIA** consumindo qualquer endpoint público de texto.

Checklist de Entrega e Avaliação

O trabalho será avaliado em duas dimensões: **(A) a aplicação funciona e reproduz o App Aluno original** (requisitos funcionais) e **(B) os conceitos de React foram aplicados corretamente** (requisitos técnicos). A nota é distribuída da seguinte forma:

A) Funcionalidade — a aplicação entrega o que o original entrega (55%)

Categoria	Requisito	Pontos	Referência (Aula / E-book)
Fluxo de Autenticação	Telas de Login, Cadastro (passos 1 e 2), Recuperar senha e Nova senha funcionando, com formulários controlados e validação (mensagem de erro na tela).	20%	RF 1, 7, 8 · Aula 12/13
Telas Internas e Navegação	Dashboard, Disciplinas, Perfil e TutorIA acessíveis e navegáveis como SPA; menu com indicação da rota ativa; redirecionamento após o login.	20%	RF 2, 3, 4, 5 · Aula 14
Consumo de Dados / API	Pelo menos uma tela busca dados em uma API externa e trata corretamente os estados de carregando / erro / dados; lista de itens renderizada a partir de uma coleção.	15%	RF 4, 6 · Aula 13

B) Implementação Técnica — os conceitos foram aplicados corretamente (45%)

Categoria	Requisito	Pontos	Referência (Aula / E-book)
Setup & Estrutura	Projeto inicializado com Vite e organizado em pastas (<code>components</code> , <code>pages</code> , <code>context</code> , <code>services</code>).	5%	Aula 10 — <code>npm create vite@latest</code>
Componentização	UI dividida em múltiplos componentes reutilizáveis, com uso de props e children .	10%	Aula 11/12 — Componentes, Props, children
Estado, Hooks e Listas	Uso correto de <code>useState</code> / <code>useEffect</code> , renderização condicional (<code>&&</code>) e <code>.map()</code> com prop key .	10%	Aulas 11/13 — Hooks, <code>&&</code> , repetição
Roteamento (React Router)	<code>BrowserRouter</code> , <code>Routes</code> , <code>Route</code> , <code>NavLink</code> e <code>useNavigate</code> configurados corretamente.	5%	Aula 14 — React Router
Context API	Dados do usuário compartilhados entre telas via Context (sem <i>prop drilling</i>).	5%	Aula 14 — Context API
API com <code>fetch</code>	<code>fetch</code> com <code>try / catch / finally</code> e checagem de <code>res.ok</code> antes de parsear.	5%	Aula 13 — Fetch, estados da requisição
Estilização / Mobile	Estilo fiel à referência (<code>className</code> + CSS) e layout responsivo (desktop e mobile).	5%	Aula 11 — Estilos (<code>.css</code>), <code>className</code>

Total: 100% (A = 55% · B = 45%)

Observação: alguns conceitos aparecem nas duas dimensões de propósito. Em **A** avalia-se se a funcionalidade existe e funciona; em **B**, se ela foi construída com a técnica correta. Ex.: o consumo de API conta em "Consumo de Dados" (funcionou?) e em "API com `fetch`" (foi feito com o padrão certo?).

Estrutura de pastas sugerida

```
app-aluno-react/  
├── public/  
├── src/  
│   ├── components/      # Componentes reutilizáveis (InputField, Card, ...)  
│   ├── pages/           # Telas/rotas (LoginPage, DashboardPage, ...)  
│   ├── context/         # UsuarioContext.jsx  
│   ├── services/        # Chamadas de API (ex.: githubService.js)  
│   ├── App.jsx  
│   ├── main.jsx  
│   └── index.css  
├── index.html  
├── package.json  
└── README.md
```

Como criar e executar o projeto

```
# 1. Criar o projeto React com Vite  
npm create vite@latest app-aluno-react  
# (escolher: React + JavaScript)  
  
# 2. Acessar a pasta  
cd app-aluno-react  
  
# 3. Instalar as dependências  
npm install  
  
# 4. Instalar o React Router  
npm install react-router-dom  
  
# 5. Executar em modo de desenvolvimento (http://localhost:5173)  
npm run dev
```

Entrega

- O projeto deve ser entregue **em dupla**.
- Submeter o **código-fonte completo** (sem a pasta `node_modules`) via **link do repositório do GitHub anexado no AVA**.
- Incluir um arquivo `README.md` claro, com instruções de como **instalar as dependências** e **executar a aplicação**.
- A avaliação N3 (Projeto Final) **não poderá ser recuperada pela prova N-1**.

Apresentação

- O projeto deve estar **inicializado e rodando na máquina** para a demonstração das funcionalidades no dia da apresentação em aula.
- Cada dupla deverá demonstrar o fluxo completo: login → navegação entre telas → consumo de API →

responsividade.

Cronograma (Aulas ABP)

Data	Aula	Atividade	Produção
08/06/2026	Aula 16 — ABP I	Produção	Parcial 1 — Esqueleto navegável + Autenticação
15/06/2026	Aula 17 — ABP II	Produção	Parcial 2 — App funcional: telas internas, estado global e API
22/06/2026	Aula 18 — ABP III	Entrega e Apresentação	Entrega Final (Parcial 1 e 2) — Projeto completo + apresentação

Entregas Parciais (Checkpoints)

Para evitar acúmulo na última aula, o projeto é construído de forma **incremental**. Cada aula de produção tem uma entrega obrigatória que serve de checkpoint e ponto de feedback. As entregas são cumulativas: a Parcial 2 incorpora e evolui a Parcial 1, e a Entrega Final consolida tudo.

Parcial 1 — Início na aula 16 (ABP I): *Esqueleto navegável + Autenticação*

Foco: **estrutura, componentes, rotas e o fluxo de autenticação**. Ao final desta aula, o projeto deve "andar" como SPA, mesmo que as telas internas ainda estejam vazias.

- ☐ Projeto criado com **Vite** e rodando (`npm run dev`), com `react-router-dom` instalado.
- ☐ **Estrutura de pastas** organizada (`components` , `pages` , `context` , `services`).
- ☐ **Todas as rotas** declaradas (`BrowserRouter` , `Routes` , `Route`) e navegáveis — as telas internas podem ser *placeholders* nesta etapa.
- ☐ **Telas de autenticação** construídas em JSX como componentes: Login, Cadastro (passos 1 e 2), Recuperar senha e Nova senha.
- ☐ **Formulários controlados** (`value` + `onChange` com `useState`) ao menos no Login e no Cadastro.
- ☐ **Componentes reutilizáveis** básicos criados (ex.: `InputField` , `Botao` , `Card`).
- ☐ **Estilização inicial** aplicada com `className` /CSS aproximando do layout original.

Cobre principalmente os blocos: *Setup & Estrutura*, *Componentização*, *Roteamento* e o início de *Estado/Formulários* e *Estilização*.

Parcial 2 — Início na aula 17 (ABP II): *App funcional: telas internas, estado global e API*

Foco: **fazer a aplicação funcionar de ponta a ponta** — dados do usuário, telas internas, consumo de API e validações.

- ☐ **Login redireciona** para o Dashboard via `useNavigate` .
- ☐ **Context API** funcionando: nome do usuário compartilhado e exibido no Dashboard, Perfil e Sidebar (sem *prop drilling*).
- ☐ **Telas internas com conteúdo real**: Dashboard, Disciplinas, Perfil e TutorIA.
- ☐ **Disciplinas** renderizada a partir de uma coleção com `.map()` e prop `key` .
- ☐ Ao menos uma tela **consumindo API** com `fetch` e os três estados **carregando / erro / dados** (`try / catch / finally` , checagem de `res.ok`).
- ☐ **Validação de formulário** com mensagem de erro via renderização condicional (`&&`).

- ☐ **NavLink** com indicação visual da rota ativa.
- ☐ **Responsividade** em andamento (desktop e mobile).

Cobre principalmente os blocos: Telas Internas e Navegação, Context API, Consumo de Dados/API e Validação de formulários.

Entrega Final — Aula 18 (ABP III): *Projeto completo + apresentação*

Pontos da avaliação: **polimento, fidelidade e entrega.**

- ☐ Atendimento dos requisitos funcionais e técnicos (Parcial 1 e 2).
- ☐ Fidelidade visual completa às telas de referência (telas/).
- ☐ Responsividade finalizada em desktop e mobile.
- ☐ `README.md` com instruções de instalação e execução.
- ☐ Código-fonte completo (**sem** `node_modules`) com link do repositório anexado no AVA.
- ☐ Projeto rodando na máquina para a apresentação.